

Despliegue de Aurora Financiera en EC2 (Apache + Gunicorn + HTTPS)

Guía paso a paso para desplegar la app **Django** en una instancia EC2 (Ubuntu) con Apache2 como *reverse proxy* hacia Gunicorn, estáticos servidos desde `/var/www/html/static`, y HTTPS con Let's Encrypt (Certbot).

Arquitectura final:



- Repo: <https://github.com/sergiorueda9090/n8n.git> (público)
- Código Django en la EC2: `/var/www/aurora`
- Estáticos públicos: `/var/www/html/static`

Dominio vs. IP: los ejemplos usan `sruedadev.com`. Reemplázalo por tu propio dominio (registro A → IP de la EC2) o, si aún no tienes dominio, por la **IP pública** de tu EC2 (en ese caso te quedas en HTTP y saltas el paso 10).

⚠ 0. Seguridad ANTES de empezar

El repo es **público**, así que cualquiera puede ver el código. Estos archivos **NUNCA** deben subir a git (ya están en `.gitignore`, verifícalo):

- `.env` → contiene `SECRET_KEY` y credenciales.
- `n8n-bedrock-ocr_accessKeys.csv` → **contiene claves AWS**. Si alguna vez se expuso, **rótala/desactívala en la consola de AWS IAM**.

En producción se crea un `.env` nuevo directamente en la EC2 (paso 5). La `SECRET_KEY` de producción debe ser **distinta** a la local.

1. Confirmar que el código está en GitHub

El repo ya existe y es público; tu rama `main` local debería estar sincronizada. Desde tu máquina, verifica que no queda nada por subir:

```
cd /home/srueda/Documentos/linux/aurora-financiera
git status          # "working tree clean" y "up to date with
origin/main"
git push            # por si tienes commits locales pendientes
```

Al ser público, la EC2 clona por HTTPS **sin credenciales**: no hace falta Deploy Key ni token.

2. Entrar a la instancia

```
ssh -i ~/.ssh/TU_LLAVE.pem ubuntu@IP_EC2
```

(Ubuntu AMI usa el usuario **ubuntu**. Usa la Elastic IP si tienes.)

Asegúrate de que el **Security Group** de la EC2 tiene abiertos los puertos **22** (SSH), **80** (HTTP) y **443** (HTTPS).

3. Instalar dependencias del sistema (en la EC2)

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip apache2 git

# Módulos de Apache necesarios para el proxy y el header HTTPS:
sudo a2enmod proxy proxy_http headers ssl rewrite
sudo systemctl restart apache2
```

4. Clonar el código en `/var/www/aurora`

Repo público → clonado directo por HTTPS. Lo dejamos con dueño **ubuntu** para que luego **git pull** actualice sin fricción:

```
sudo mkdir -p /var/www/aurora /var/www/html/static
sudo chown $USER:$USER /var/www/aurora /var/www/html/static
git clone https://github.com/sergiorueda9090/n8n.git /var/www/aurora
```

5. Entorno virtual + dependencias de Python

```
cd /var/www/aurora
python3 -m venv env
./env/bin/pip install --upgrade pip
./env/bin/pip install -r requirements.txt
```

Si usarás **SQLite** (opción simple) puedes quitar **psycopg2-binary** del **requirements.txt**. Si usarás **PostgreSQL/Supabase**, déjalo.

6. Crear el `.env` de producción

```
cp /var/www/aurora/.env.produccion.example /var/www/aurora/.env
# Genera una SECRET_KEY nueva:
./env/bin/python -c "import secrets;print(secrets.token_urlsafe(50))"
nano /var/www/aurora/.env
```

Rellena como mínimo:

- `SECRET_KEY` → la que acabas de generar.
- `DEBUG=False`
- `ALLOWED_HOSTS=sruedadev.com,www.sruedadev.com,127.0.0.1` (o `IP_EC2,127.0.0.1` si vas por IP)
- `CSRF_TRUSTED_ORIGINS=https://sruedadev.com,https://www.sruedadev.com` (si vas por IP+HTTP: `http://IP_EC2`)
- `STATIC_ROOT=/var/www/html/static`
- Base de datos (SQLite por defecto ya viene lista; Postgres/Supabase en la plantilla).
- `N8N_WEBHOOK_URL` → usa la URL de **producción** (`/webhook/aura-web`, no la de test).

7. Migraciones + recopilar estáticos

```
cd /var/www/aurora
./env/bin/python manage.py migrate
./env/bin/python manage.py collectstatic --noinput # ->
/var/www/html/static
```

(Opcional) crea un superusuario para `/admin/`:

```
./env/bin/python manage.py createsuperuser
```

8. Permisos

El código lo maneja `ubuntu` (dueño del clon) y Gunicorn corre como `ubuntu`. Apache solo necesita **leer** los estáticos:

```
chmod 600 /var/www/aurora/.env # solo tu usuario lo lee
chmod -R a+rX /var/www/html/static # Apache (www-data) puede leerlos
```

9. Servicio Gunicorn (systemd)

```
sudo cp /var/www/aurora/deploy/gunicorn.service \
    /etc/systemd/system/gunicorn-aurora.service

sudo systemctl daemon-reload
sudo systemctl enable --now gunicorn-aurora
sudo systemctl status gunicorn-aurora    # debe verse "active (running)"
```

Prueba local de que Django responde:

```
curl -I http://127.0.0.1:8001/    # debe devolver HTTP 200/302
```

10. Configurar Apache (VirtualHost)

```
sudo cp /var/www/aurora/deploy/aurora-apache.conf \
    /etc/apache2/sites-available/aurora.conf

# Ajusta ServerName / ServerAlias a tu dominio o IP:
sudo nano /etc/apache2/sites-available/aurora.conf

sudo a2ensite aurora.conf
sudo a2dissite 000-default.conf    # desactiva el sitio por defecto
sudo apache2ctl configtest        # debe decir "Syntax OK"
sudo systemctl reload apache2
```

Comprueba en el navegador <http://srudedev.com> (o http://IP_EC2) — ya debería cargar la landing con estilos e imágenes.

11. HTTPS con Certbot (Let's Encrypt) — *solo si tienes dominio*

Si por ahora usas solo la **IP** de la EC2, **omite este paso** y quédate en HTTP (con `CSRF_TRUSTED_ORIGINS=http://IP_EC2` en el `.env`).

```
sudo apt install -y certbot python3-certbot-apache
sudo certbot --apache -d srudedev.com -d www.srudedev.com
```

Elige **redirección automática HTTP → HTTPS** cuando lo pregunte. Certbot crea el bloque `<VirtualHost *:443>` con el certificado.

11.1 Ajuste importante tras Certbot

Edita el vhost SSL que generó Certbot y asegúrate de que el header enviado a Django sea **https** (para que valide CSRF del chatbot y genere URLs seguras):

```
sudo nano /etc/apache2/sites-available/aurora-le-ssl.conf
```

Dentro del `<VirtualHost *:443>`, deja/añade:

```
RequestHeader set X-Forwarded-Proto "https"
```

Luego:

```
sudo apache2ctl configtest && sudo systemctl reload apache2
```

Renovación automática (ya viene por defecto). Verifícala:

```
sudo certbot renew --dry-run
```

12. Verificación final

- <https://srudedev.com> carga con candado ✓ (o http://IP_EC2 si vas por IP)
- Imágenes y CSS cargan (vienen de `/static/`) ✓
- El chatbot **Aurorita** responde (prueba escribir un mensaje) — esto valida que `POST /api/chat/` → n8n funciona con CSRF/HTTPS ✓
- Al **cerrar y reabrir** el chat, arranca una conversación nueva (nuevo `chat_id`) ✓
- <https://srudedev.com/admin/> abre el admin ✓

13. Actualizaciones futuras (redploy con git)

En tu máquina, haz commit y push de los cambios:

```
git add -A && git commit -m "... " && git push
```

En la EC2, baja los cambios y recarga (todo como `ubuntu`, que es el dueño del clon):

```
cd /var/www/aurora
git pull
./env/bin/python manage.py migrate
./env/bin/python manage.py collectstatic --noinput
sudo systemctl restart gunicorn-aurora
```

14. Solución de problemas

Síntoma	Dónde mirar / Causa probable
Error 500	<code>sudo journalctl -u gunicorn-aurora -n 50</code> — falta una var en <code>.env</code> o dependencia
CSS/imágenes rotas	¿Corriste <code>collectstatic</code> ? ¿ <code>STATIC_ROOT=/var/www/html/static</code> ? ¿permisos <code>www-data</code> ?
502 Bad Gateway	Gunicorn caído: <code>sudo systemctl status gunicorn-aurora</code>
Chatbot da 403	Falta <code>CSRF_TRUSTED_ORIGINS</code> o el header <code>X-Forwarded-Proto https</code> en el <code>vhost 443</code>
<code>DisallowedHost</code>	Añade el dominio/IP a <code>ALLOWED_HOSTS</code> en <code>.env</code>
Chatbot no conecta	<code>N8N_WEBHOOK_URL</code> apunta a <code>/webhook-test/</code> (solo dev). Usa <code>/webhook/aura-web</code>
Logs de Apache	<code>sudo tail -f /var/log/apache2/aurora_error.log</code>

Resumen de puertos / rutas

Elemento	Valor
Repo	<code>https://github.com/sergiorueda9090/n8n.git</code> (público)
Código Django	<code>/var/www/aurora</code>
Virtualenv	<code>/var/www/aurora/env</code>
Estáticos	<code>/var/www/html/static</code>
Gunicorn	<code>127.0.0.1:8001</code> (interno)
Apache	<code>:80</code> → redirige a <code>:443</code>
Servicio	<code>gunicorn-aurora.service</code>